```
"""
Project 1: Rule-Based AI Chatbot
DecodeLabs AI Internship - Batch 2026

Author: Arsalan
Description:
    A rule-based conversational chatbot
that uses dictionary-based
    intent matching (O(1) lookup) instead
of long if-elif ladders.
    The bot runs in a continuous loop,
sanitizes user input, matches
    it against a knowledge base of intents,
and falls back to a
    default response for unrecognized
input.
"""

import random
import datetime

# -------------------------------------------
----------------
# PHASE 2: KNOWLEDGE BASE
# Dictionary of intents -> possible
responses.
# Using a dictionary gives O(1) lookup
instead of an O(n)
# if-elif chain, and keeps the bot easy to
extend later.
```

```
# -----------------------------------------
----------------
knowledge_base = {
    "hello": ["Hi there! How can I help you
today?", "Hello! Nice to see you."],
    "hi": ["Hey! What's up?", "Hi! How are
you doing?"],
    "how are you": ["I'm just a bunch of
code, but I'm running smoothly! You?",
                     "Doing great, thanks
for asking!"],
    "what is your name": ["I'm ChatBuddy,
your friendly rule-based assistant."],
    "who made you": ["I was built by
Arsalan as Project 1 for the DecodeLabs AI
Internship."],
    "what can you do": ["I can chat about a
few basic things: greetings, my name, "
                         "the time, and
jokes. Type 'help' to see options."],
    "help": ["Try saying: hello, how are
you, what is your name, tell me a joke, "
              "what time is it, or bye to
exit."],
    "tell me a joke": ["Why do programmers
prefer dark mode? Because light attracts
bugs!",
                        "Why did the AI
break up with the database? Too many
relationships."],
    "what time is it": [],  # handled
dynamically below
    "thank you": ["You're welcome!",
"Anytime!"],
```

```
    "thanks": ["No problem!", "Happy to
help."],
}

# Exit commands that break the loop
exit_commands = ["bye", "exit", "quit",
"goodbye"]


def get_dynamic_response(intent):
    """Handle intents that need a computed
answer instead of a fixed string."""
    if intent == "what time is it":
        now =
datetime.datetime.now().strftime("%I:%M
%p")
        return f"Right now it's {now}."
    return None


def get_response(user_input):
    """
    Look up the cleaned user input in the
knowledge base.
    Falls back to a default 'I do not
understand' style message
    if no intent matches (using dict.get()
for a single atomic
    lookup + fallback operation).
    """
    dynamic =
get_dynamic_response(user_input)
    if dynamic:
        return dynamic
```

```
    responses =
knowledge_base.get(user_input)
    if responses:
        return random.choice(responses)

    return "I'm sorry, I do not understand
that yet. Type 'help' to see what I can
do."


def chatbot():
    """Main driver: runs the continuous
input -> process -> output loop."""
    print("=" * 55)
    print(" ChatBuddy - Rule-Based AI
Chatbot (Project 1)")
    print(" Type 'help' anytime. Type 'bye'
to exit.")
    print("=" * 55)

    while True:
        raw_input_text = input("You: ")

        # PHASE 1: Sanitization /
Normalization
        clean_input =
raw_input_text.lower().strip()

        # Exit strategy: clean break out of
the infinite loop
        if clean_input in exit_commands:
            print("ChatBuddy: Goodbye! Have
a great day. 👋")
```

```
            break

        # Skip empty input instead of
crashing / matching wrongly
        if clean_input == "":
            print("ChatBuddy: Please type
something.")
            continue

        reply = get_response(clean_input)
        print(f"ChatBuddy: {reply}")


if __name__ == "__main__":
    chatbot()
```